

LSDL Lecture 01

How to use a pre-trained model?

Ildus Sadrtidinov, 08.09.2025

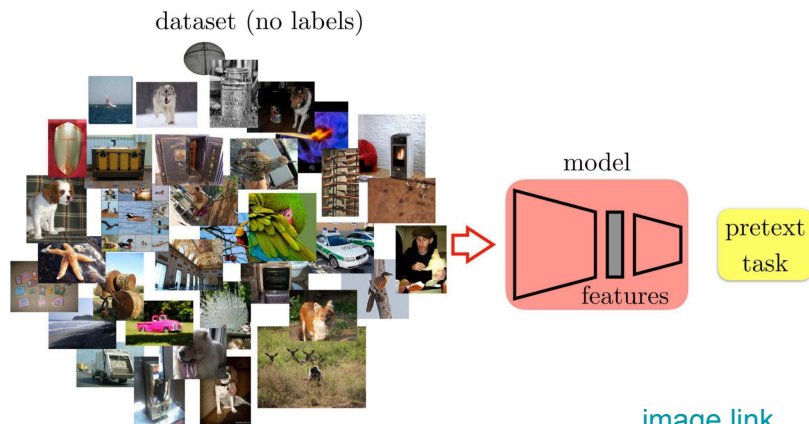
Large datasets

1) Impossible to collect large amounts of high-quality data



[image link](#)

2) Too expensive to annotate → **self-supervised learning**

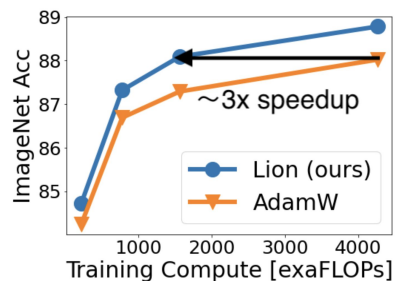


[image link](#)

Large models

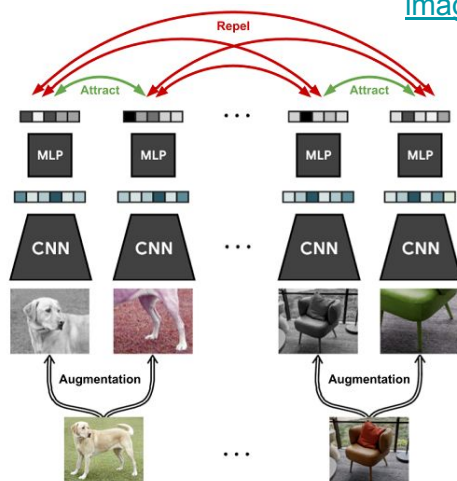
1) What training algorithms to use?

[image link](#)



optimizers

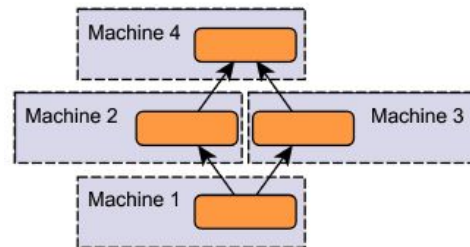
[image link](#)



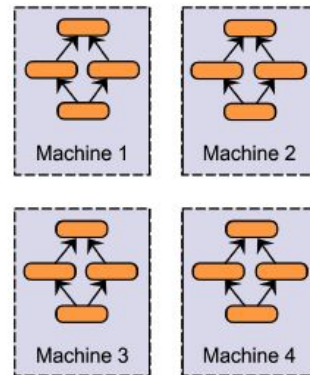
training
objectives

2) Too large to fit in a single GPU

Model Parallelism



Data Parallelism

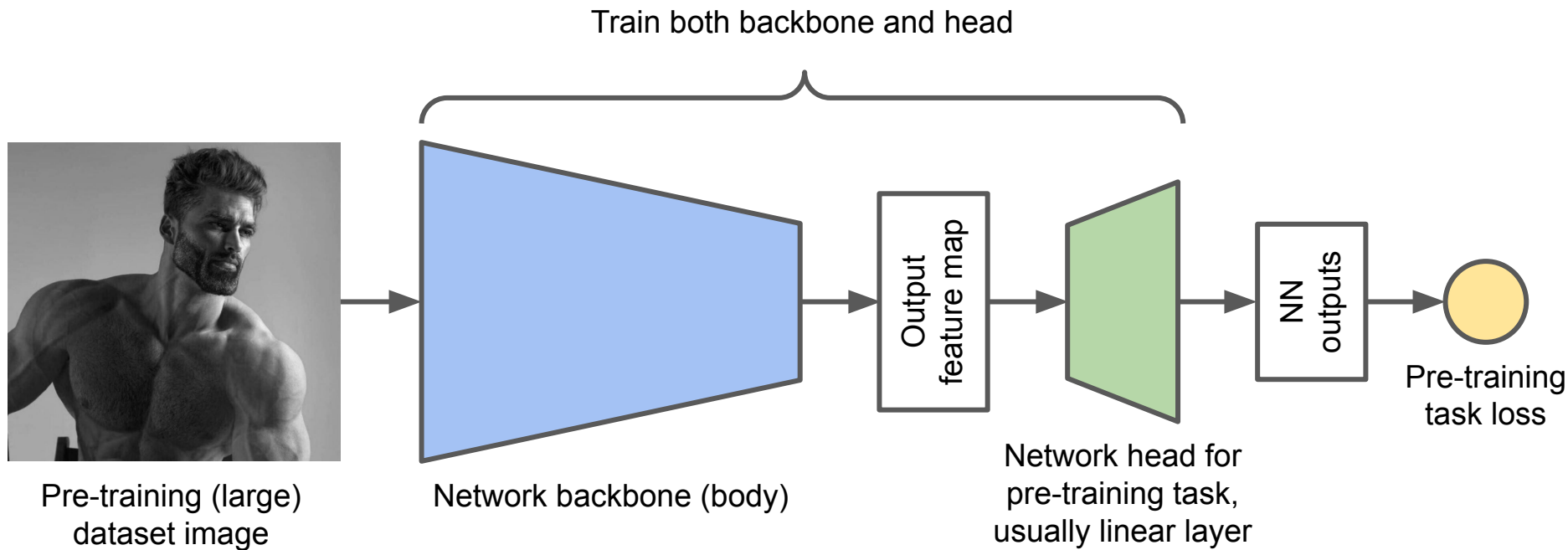


[image link](#)

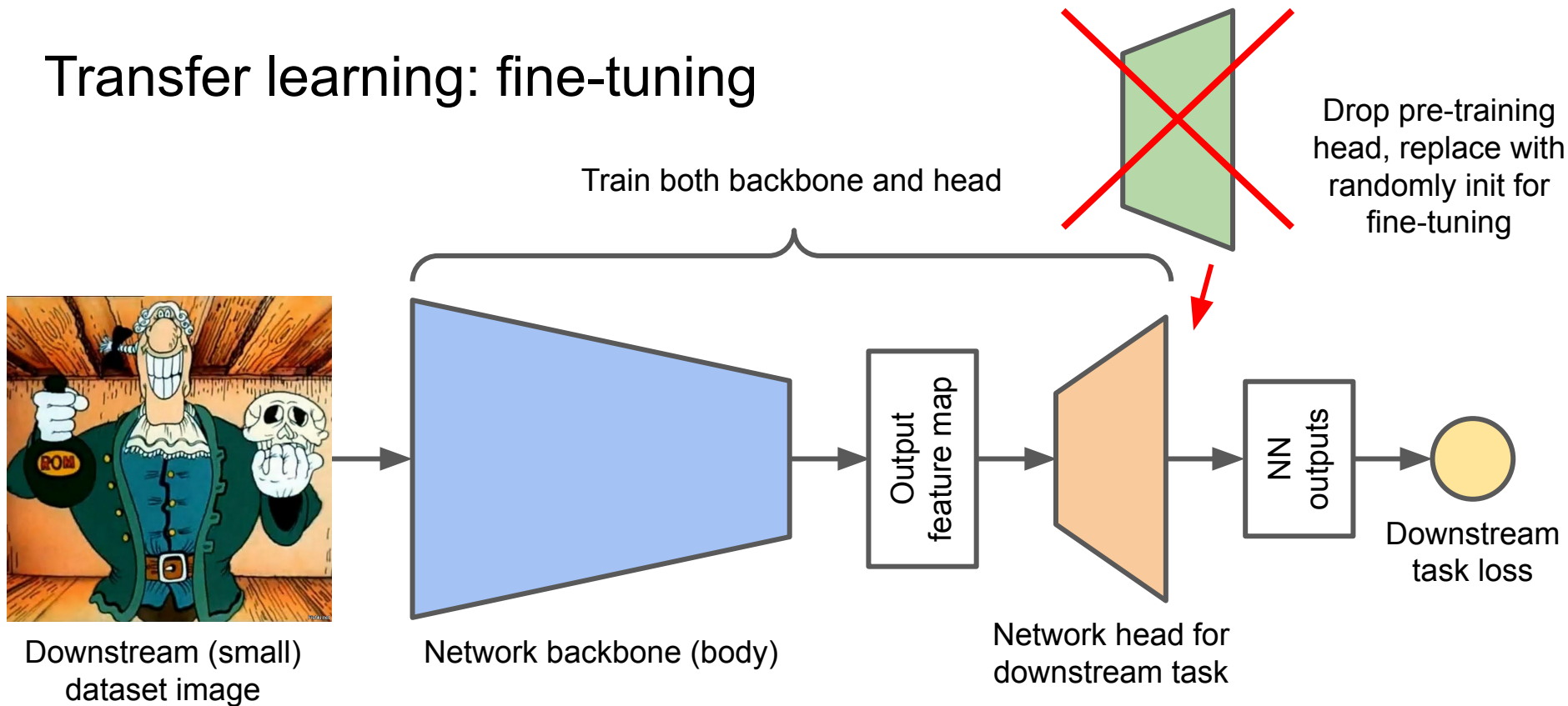
Course overview

- **Vision:** classic pre-text tasks, contrastive learning, masked image modelling
- **NLP:** Parameter-Efficient Fine-Tuning (PEFT), Retrieval Augmented Generation (RAG), Reinforcement Learning with Human Feedback (RLHF)
- **Misc:** ensembling, weight averaging, pruning, quantization, distillation
- **Audio:** contrastive & masked modelling approaches

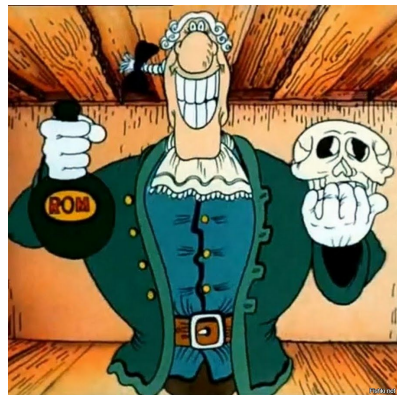
Transfer learning: pre-training



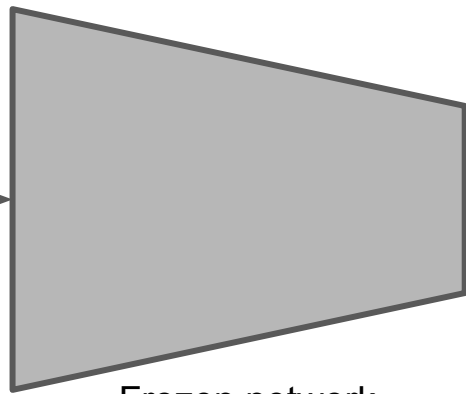
Transfer learning: fine-tuning



Transfer learning: linear probing

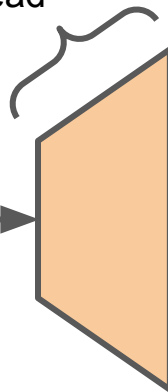


Downstream (small)
dataset image

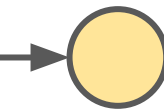


Frozen network
backbone (body)

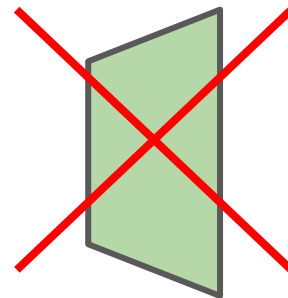
Train only head



Network head for
downstream task



Downstream
task loss



Drop pre-training
head, replace with
randomly init for
linear probing

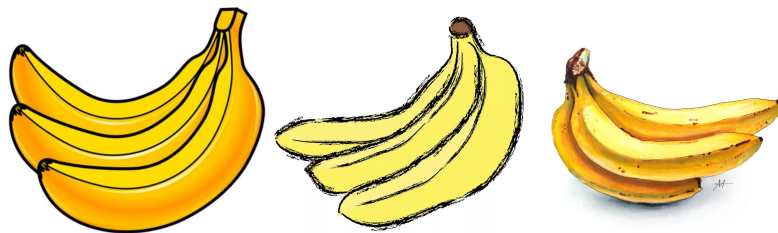
When to use fine-tuning or linear probing?

- Fine-tuning → better **in-distribution (ID)** accuracy
- Linear probing → better **out-of-distribution (OOD)** accuracy

In-distribution



Out-of-distribution



Fine-tuning distorts pre-trained features

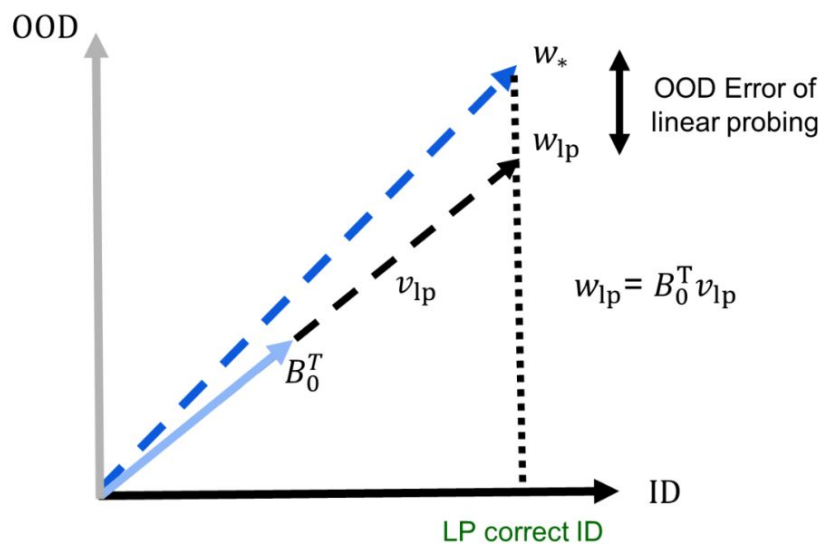
$$\hat{L}(v, B) = \|XB^\top v - Y\|_2^2$$

$$\nabla_B \hat{L}(v, B) = 2v(Y - XB^\top v)^\top X$$

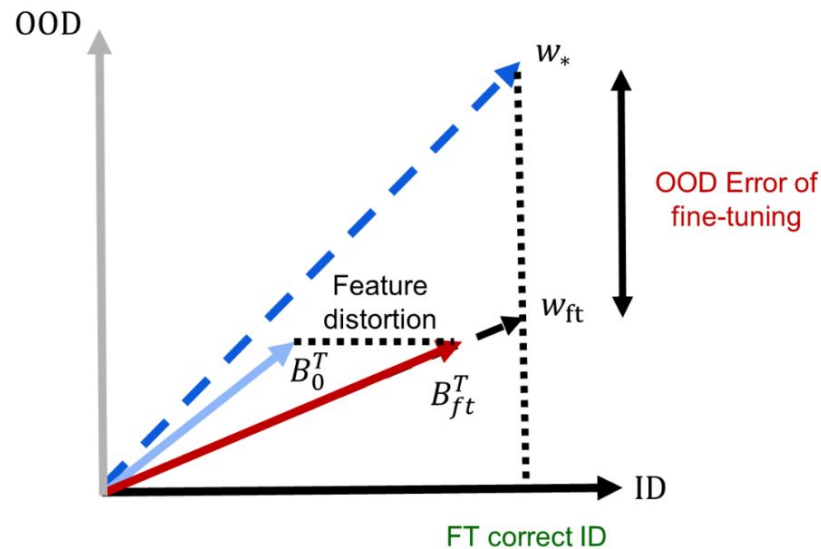
$$\nabla_B \hat{L}(v, B)u = 0$$

$$B^{k+1}u = (B^k - \eta \nabla_{B^k} \hat{L}(v, B^k))u = B^k u$$

Fine-tuning distorts pre-trained features



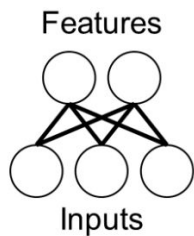
(a) Toy example (Linear probing)



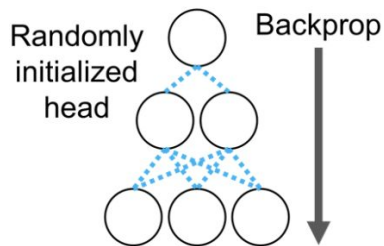
(b) Toy example (fine-tuning)

Fine-tuning distorts pre-trained features

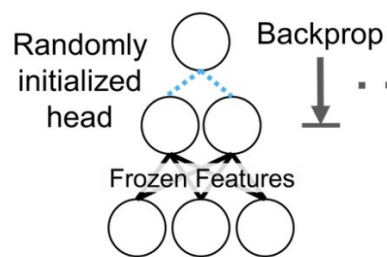
Pretraining



(a) Fine-tuning

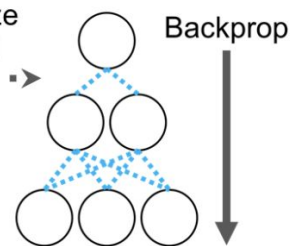


(b) Linear probing



Initialize head

(c) LP-FT



ID test

85.1%

82.9%

85.7%

OOD test

59.3%

66.2%

68.9%

Average accuracies (10 distribution shifts)

Fine-tuning distorts pre-trained features

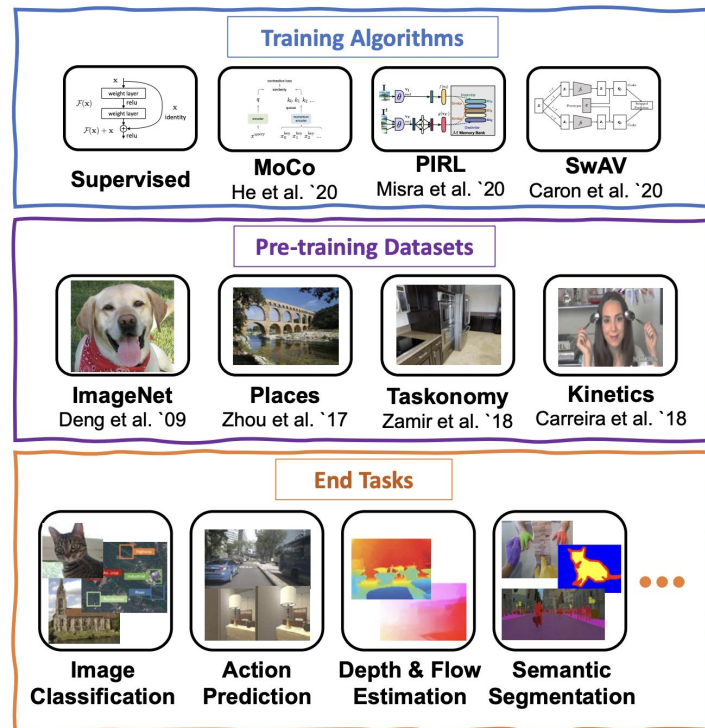
- Early stopping does not mitigate feature distortion
- ID features are distorted more than OOD features

	ID	OOD
FT	0.0188 ± 0.0001	0.0167 ± 0.0001
LP-FT	0.0011 ± 0.0001	0.0009 ± 0.0001

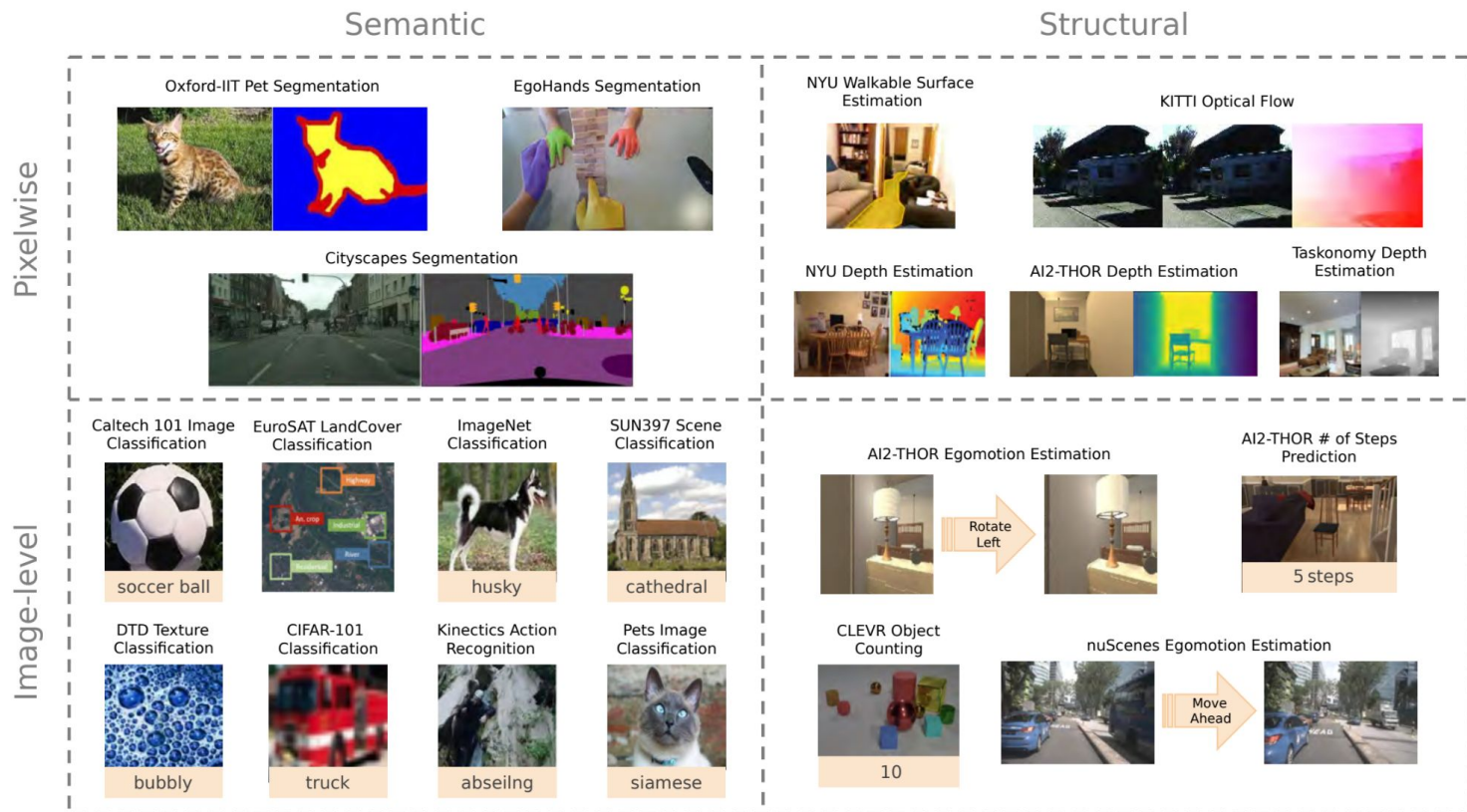
- Pre-trained features must be suitable for the task
- If ID $\approx$ OOD, then FT is better than LP

Which pre-training task to choose?

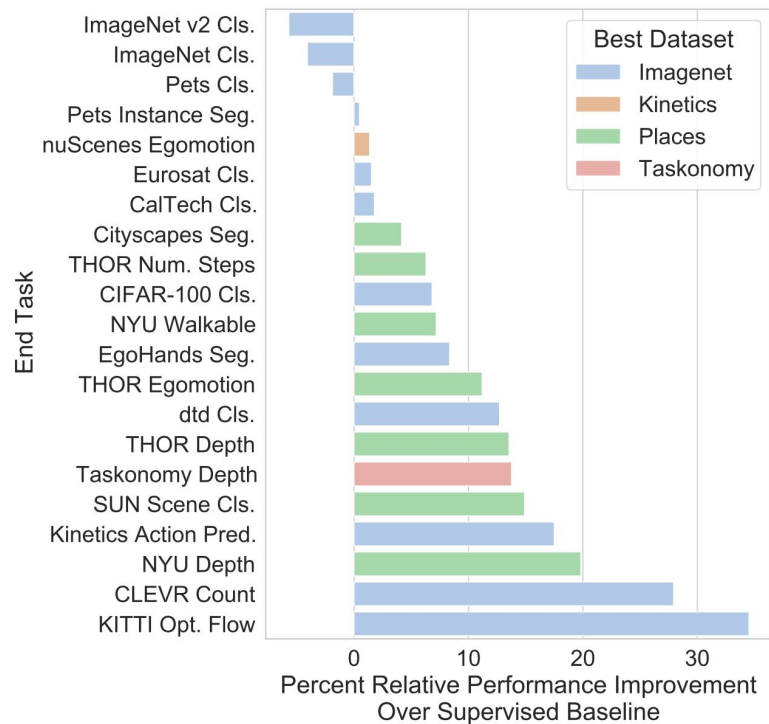
- Supervised or self-supervised?
- Which dataset to use for pre-training?
- Which network architecture to use?*



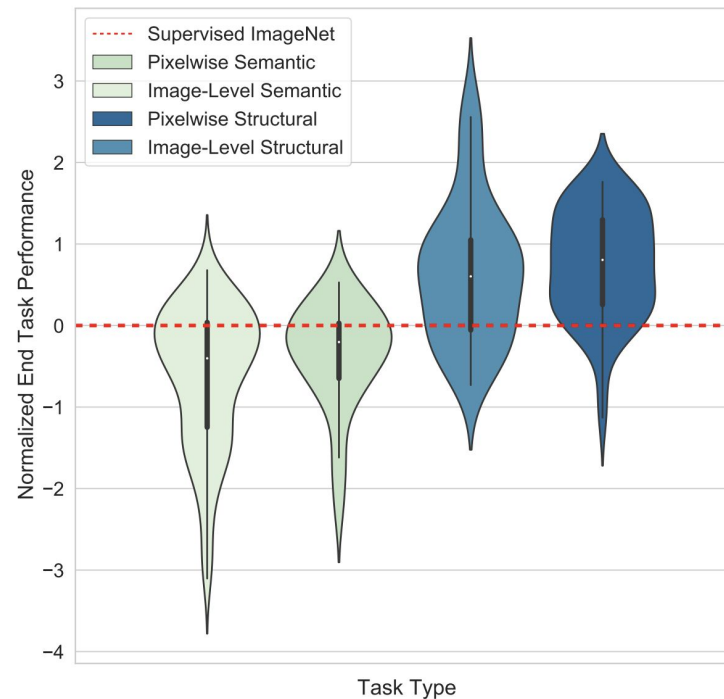
Categorization of downstream tasks



Supervised vs. self-supervised

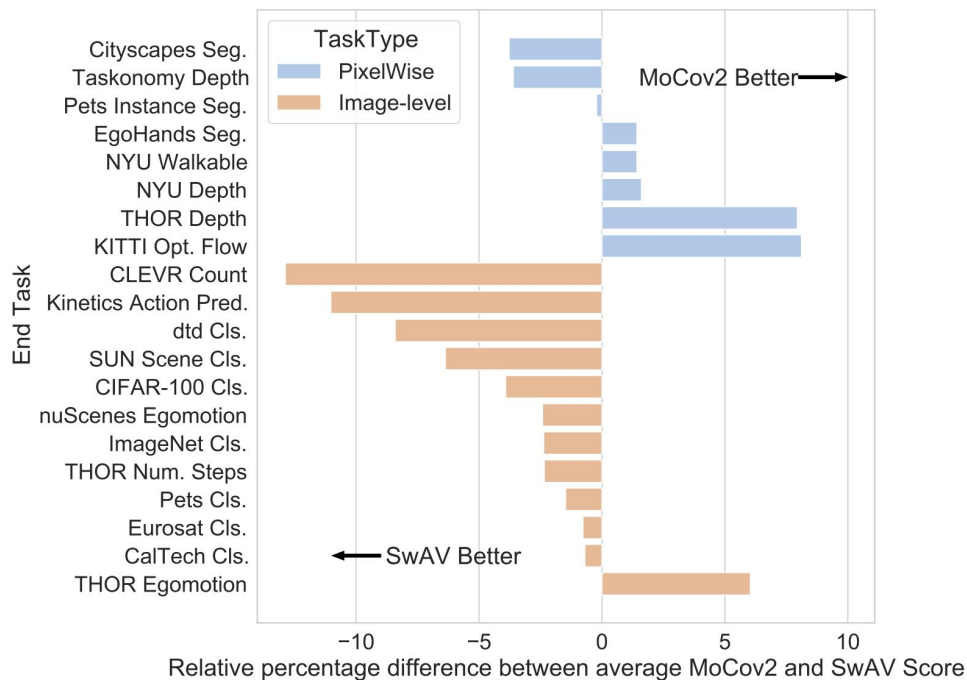


Comparison on different datasets



Comparison on different tasks

Self-supervised vs. self-supervised



**Comparison of SwAV and MoCov2
(two self-supervised algorithms)**

Is PT accuracy a good proxy for downstream quality?

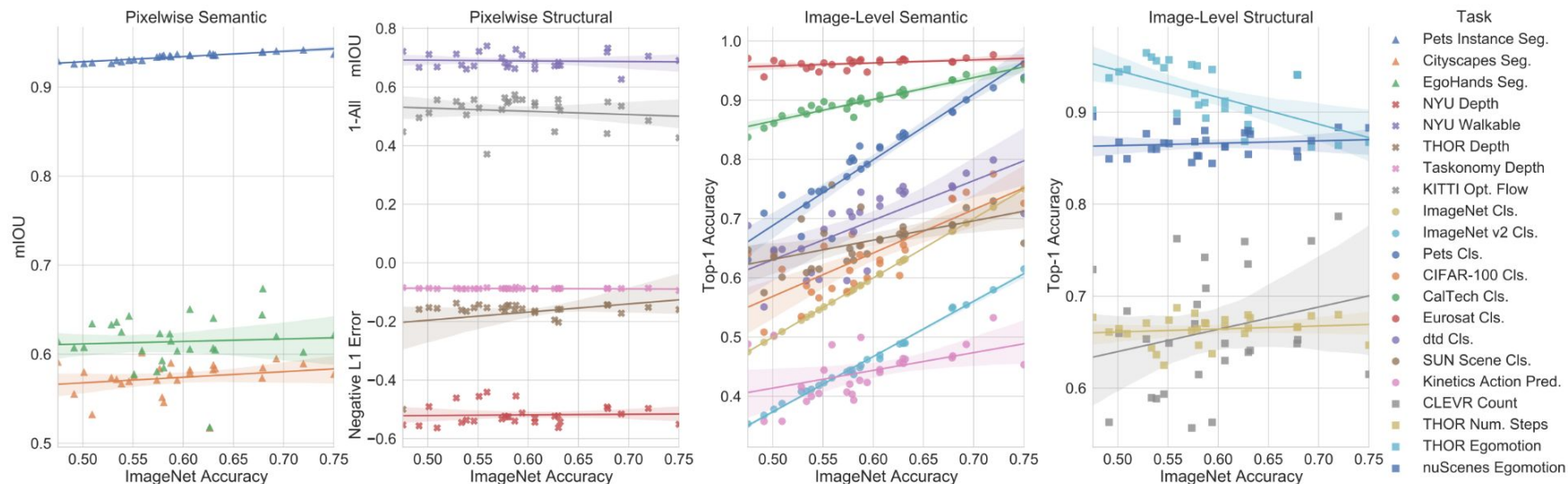


Figure 4. Correlation of end task performances with ImageNet classification accuracy. The plots show the end task performance against the ImageNet top-1 accuracy for all end tasks and encoders. Each point represents a different encoder trained with different algorithms and datasets. This reveals the lack of a strong correlation between the performance on ImageNet classification and tasks from other categories.

Does the pre-training dataset matter?

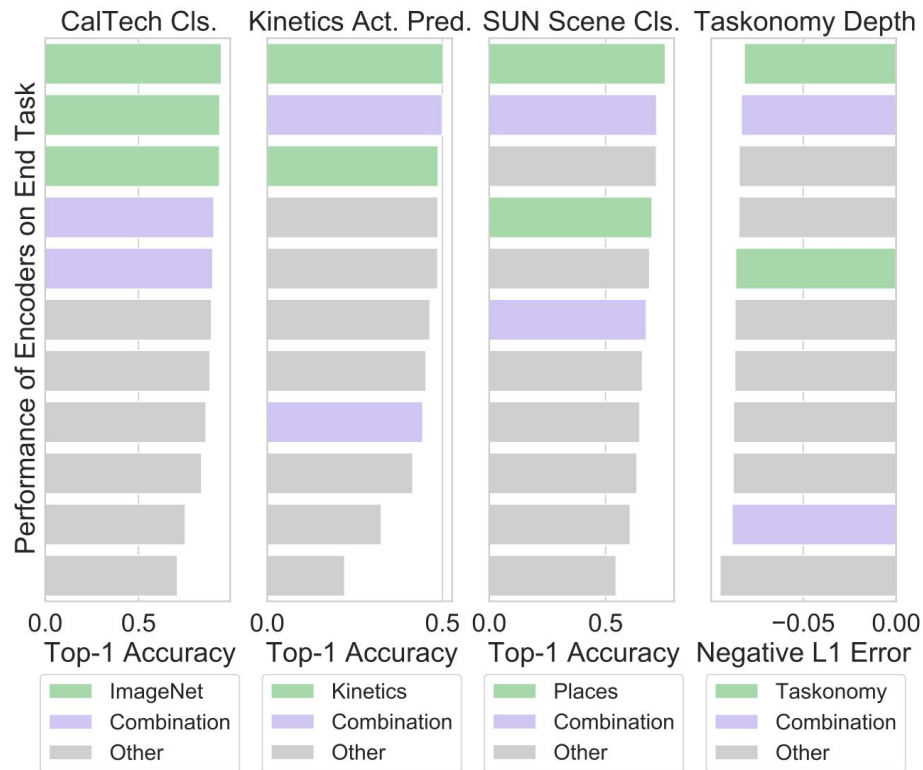
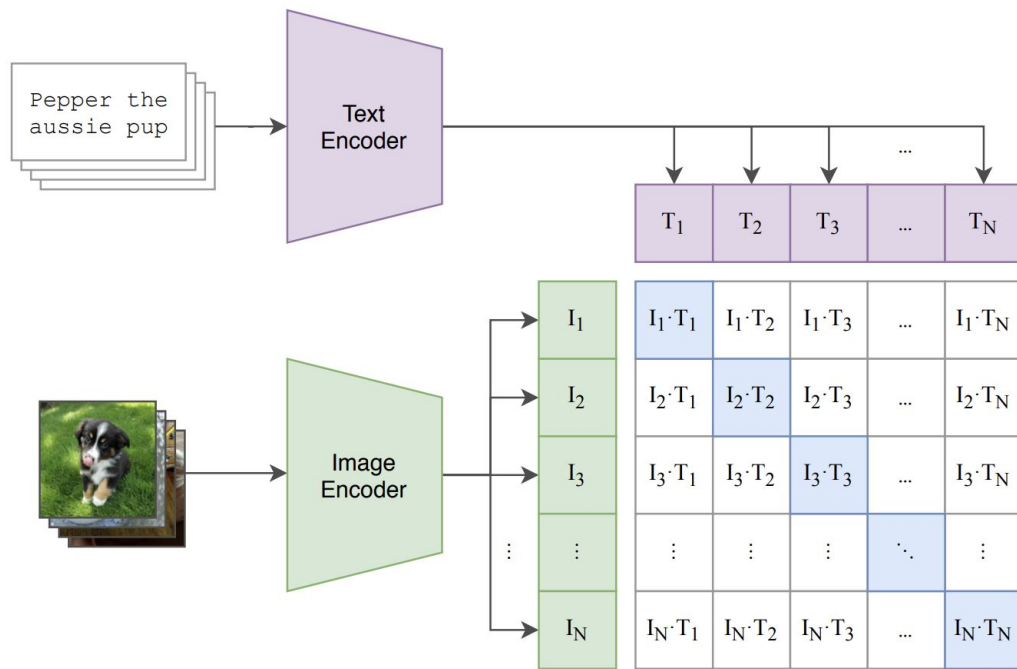


Figure 7. **Similarity of the pre-training datasets and end tasks.**

CLIP: Contrastive Language-Image Pre-training

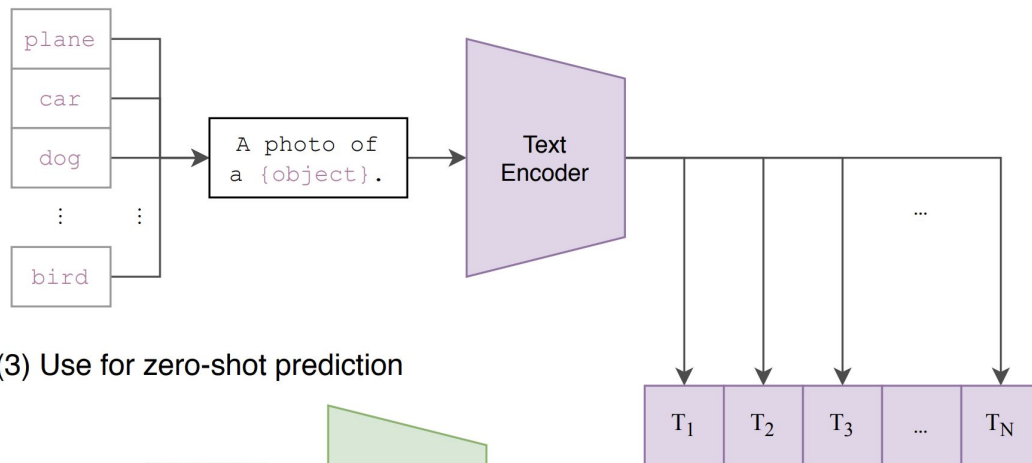
(1) Contrastive pre-training



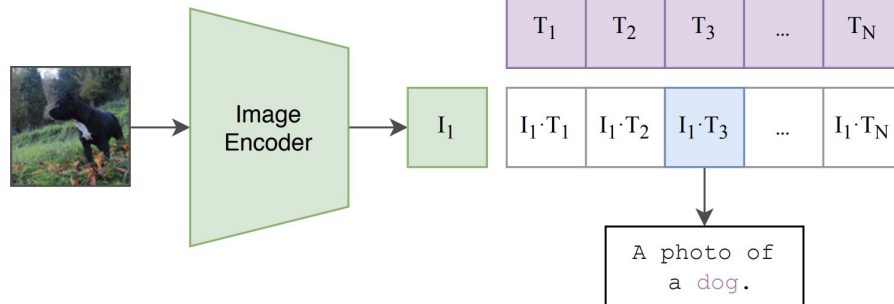
- Encode images and texts into the same embedding space
- Bring together embeddings of pairs (image, caption)
- Push apart embeddings from different pairs

CLIP: Contrastive Language-Image Pre-training

(2) Create dataset classifier from label text

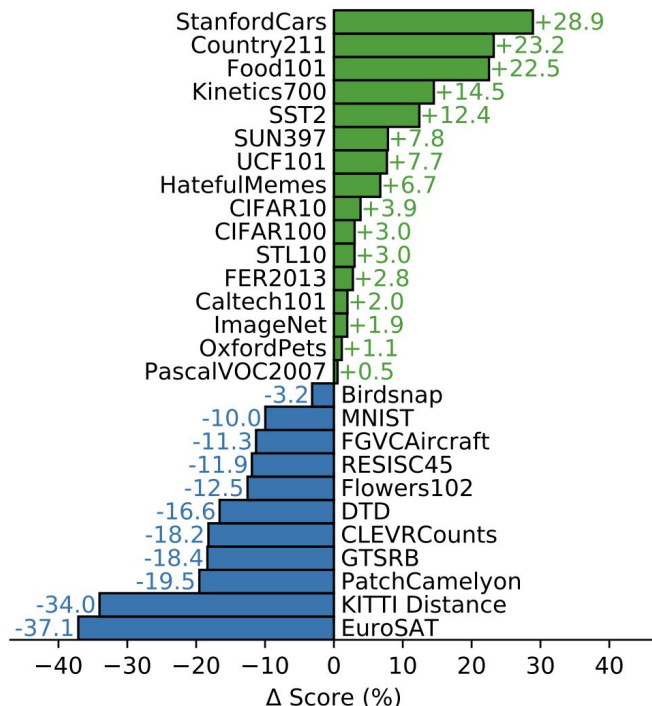


(3) Use for zero-shot prediction



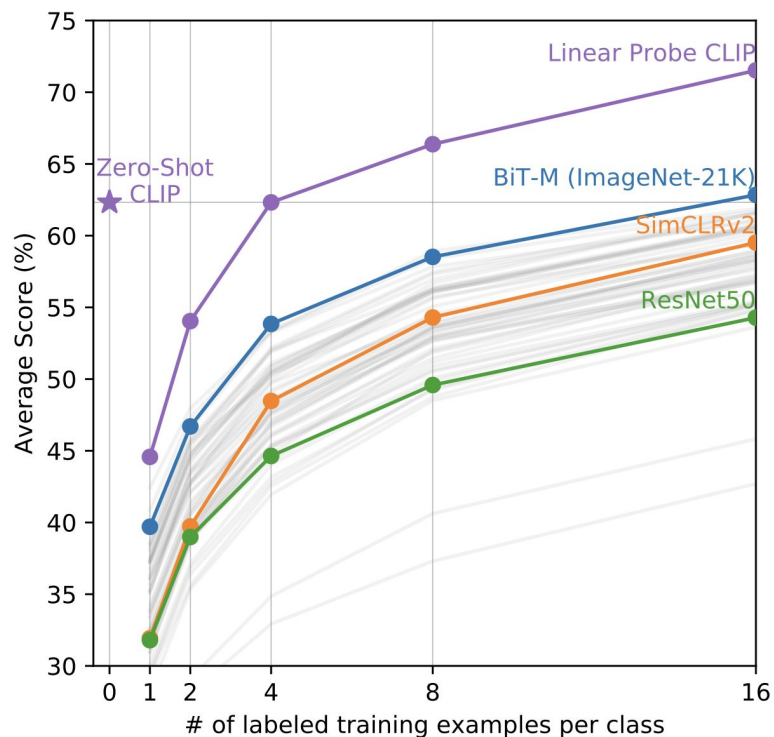
- Get image embedding
- Calculate similarity to embeddings of class texts
- Use similarity scores as classification logits

CLIP is a strong zero-shot predictor



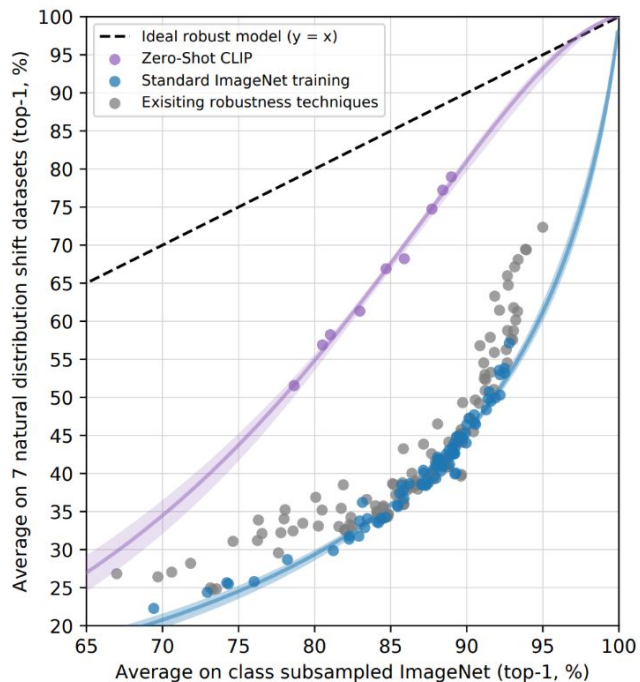
Zero-Shot CLIP vs. Linear Probe on ResNet50

Zero-shot CLIP is competitive with a fully supervised baseline



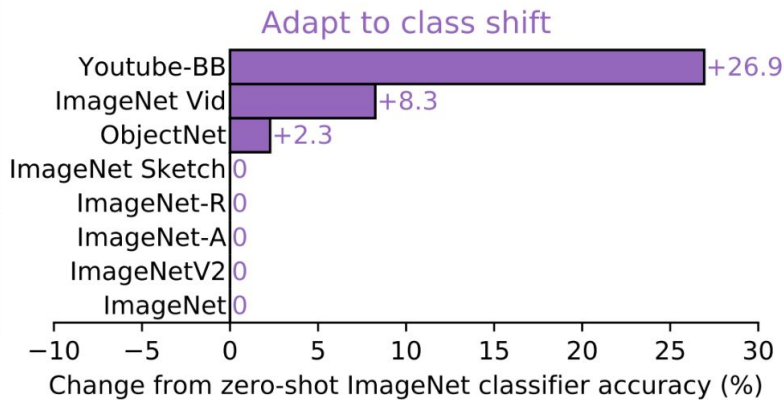
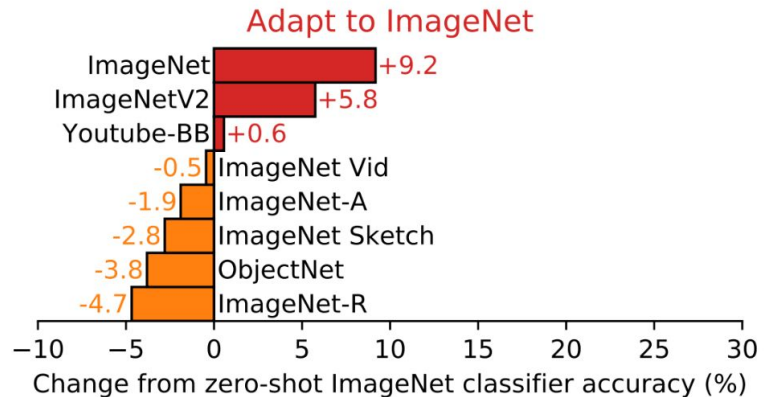
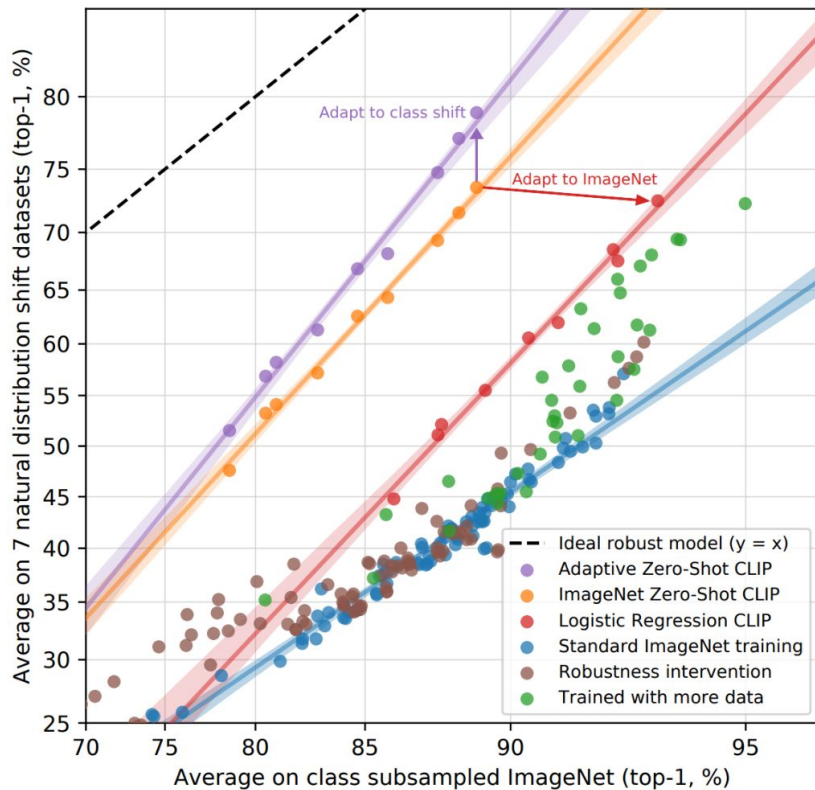
Zero-shot CLIP outperforms few-shot linear probes

Zero-shot CLIP is a robust model

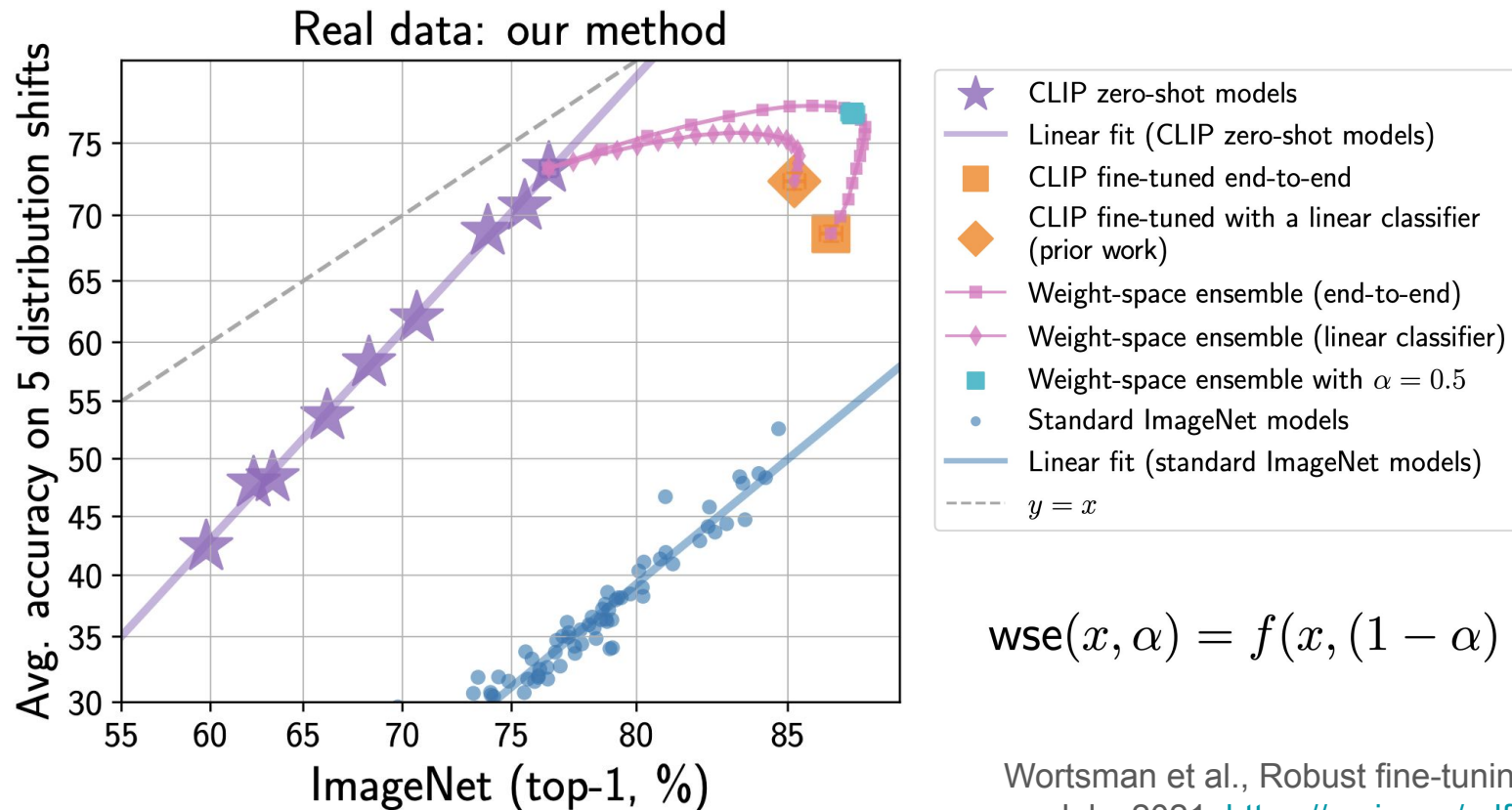


	Dataset Examples						ImageNet ResNet101	Zero-Shot CLIP	Δ Score
ImageNet							76.2	76.2	0%
ImageNetV2							64.3	70.1	+5.8%
ImageNet-R							37.7	88.9	+51.2%
ObjectNet							32.6	72.3	+39.7%
ImageNet Sketch							25.2	60.2	+35.0%
ImageNet-A							2.7	77.1	+74.4%

ID adaptation hurts robustness

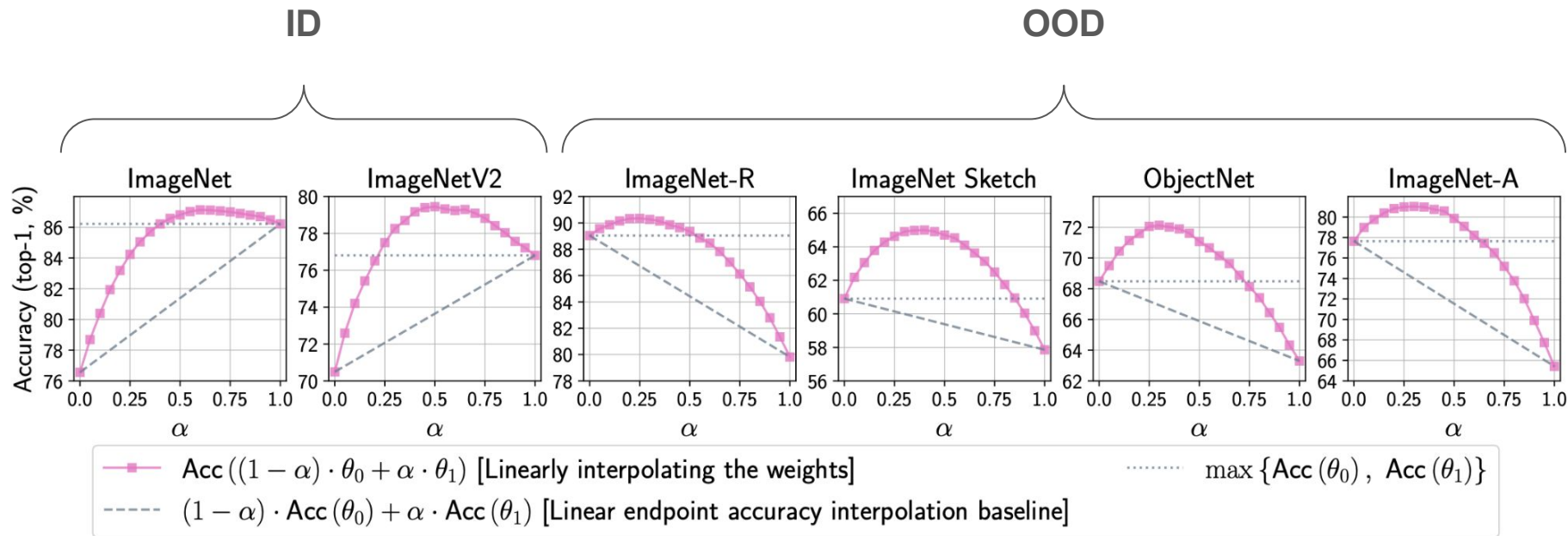


How to get both strong and robust model?

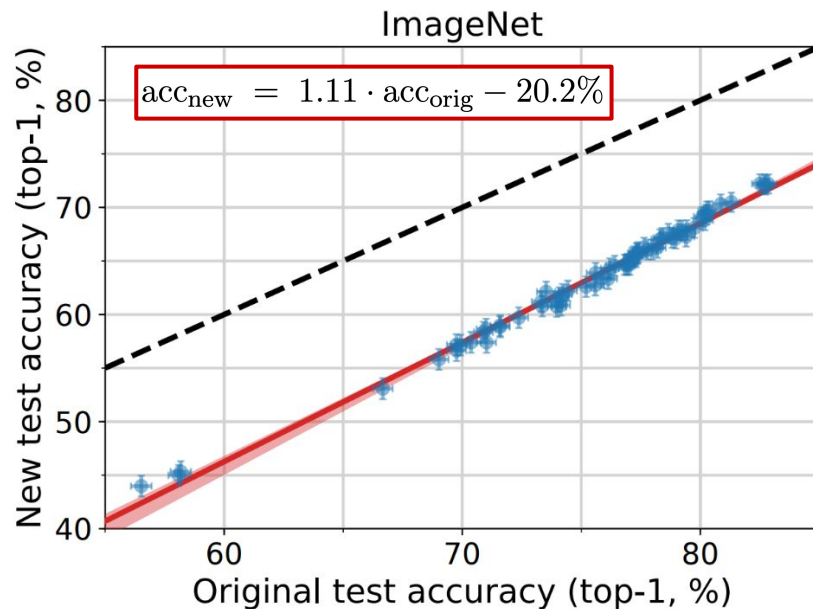
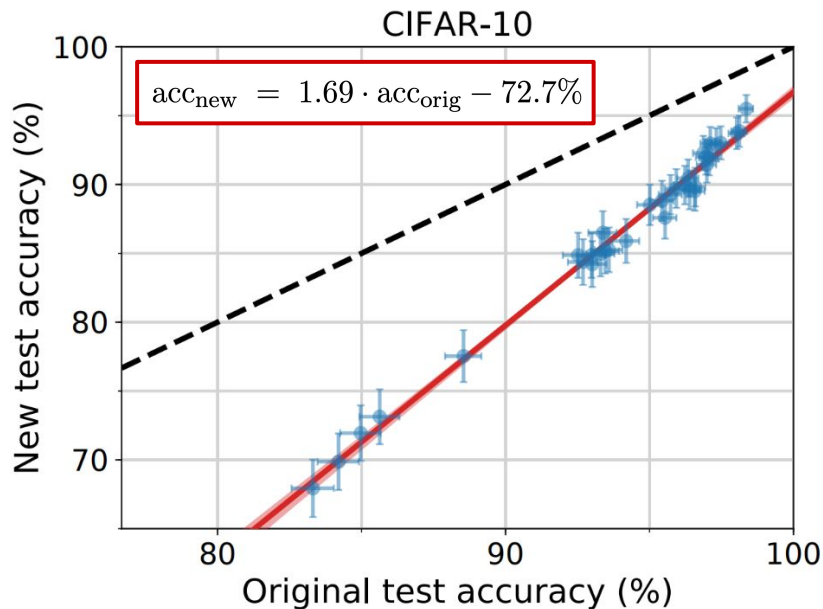


$$\text{wse}(x, \alpha) = f(x, (1 - \alpha) \cdot \theta_0 + \alpha \cdot \theta_1)$$

Interpolating between fine-tuned and zero-shot solutions



Do ImageNet Classifiers Generalize to ImageNet?



--- Ideal reproducibility

● Model accuracy

— Linear fit

Recht et al., Do ImageNet Classifiers Generalize to ImageNet?, 2019, <https://arxiv.org/pdf/1902.10811>

Do ImageNet Classifiers Generalize to ImageNet?

Validation set: 10 images/class

- **Matched frequency:** original selection frequency distribution for each class
- **Threshold0.7:** sample images with selection frequency > 0.7
- **TopImages:** select images with top frequency

Sampling Strategy	Average MTurk Selection Freq.	Average Top-1 Accuracy Change	Average Top-5 Accuracy Change
MatchedFrequency	0.73	-11.8%	-8.2%
Threshold0.7	0.85	-3.2%	-1.2%
TopImages	0.93	+2.1%	+1.8%